SYNAPSE — Technical Documentation

Team: Segfault Society | **Event:** CIPHER 2.0, IIT Sri Lanka | **Scenario:** 01 — Smart University Resource Allocation System

1. Problem

University shared resources — computer labs, meeting rooms, AV studios, and testing equipment — are allocated informally today. The resulting pathologies are well-documented in the Phase 1 Case Analysis:

- **Double-booking conflicts**: two requesters claim the same slot with no arbiter; the later arrival is turned away at the door.
- **Unfair distribution**: a small group of users monopolises prime slots over weeks; infrequent users are effectively locked out because their requests never win a race.
- **Opacity**: denied requests receive no explanation; users cannot judge whether the outcome was fair or whether an alternate slot or resource would have worked.
- **Poor utilisation**: no-shows hold confirmed slots until the session ends; the slot goes to waste while queued requesters could have used it.

These are not UI problems — they stem from the absence of a server-authoritative allocation engine. SYNAPSE is that engine.

2. Solution Overview

SYNAPSE is a **server-authoritative, conflict-free allocation engine** that sits entirely inside Postgres and is surfaced through a Next.js 16 front-end. Four mechanisms work together:

Mechanism	What it does
Conflict-free arbitration	A GiST exclusion constraint makes overlapping confirmed bookings physically impossible. A per-resource advisory lock serialises simultaneous requests so the winner is the highest-priority requester, not the fastest packet.
Priority scoring	Every booking request is scored on five dimensions. The full breakdown is returned in a Decision Explainer JSON so users see exactly why they won or were waitlisted.
Fairness rebalancing	A periodic recalculation computes each user's served hours vs. their fair share of each resource class. Under-served users receive a γ boost in the next priority calculation; over-served users receive zero.
Smart waitlist & swap	Cancellations and no-show releases atomically promote the top-ranked waiter. A swap RPC evaluates mutual-gain and, if both sides improve, reassigns the bookings in a single transaction.

The system is genuinely **server-authoritative**: RLS revokes all direct `INSERT/UPDATE/DELETE` from clients. The only write path is through `SECURITY DEFINER` RPCs that validate actor identity and enforce business rules inside a single transaction.

3. Core Logic and Algorithms

3.1 Priority Formula

Every booking request calls `priority_score(member, resource, start, end, purpose)`:

```
score = α·urgency + β·role_weight + γ·fairness_deficit − δ·recency_penalty + ε·academic_purpose
```

All five weights are stored in `policy_settings` and are live-editable by an admin. Defaults: α=0.25, β=0.30, γ=0.30, δ=0.10, ε=0.05.

Component definitions (all clamped 0–1):

Component	Formula
`urgency`	`max(0, 1 − hours_until_start / 168)` + 0.15 sharpening boost when start ≤ 48 h away
`role_weight`	Policy lookup: faculty/lab_manager/admin → 1.0; final-year student → 0.8; year ≥ 4 → 0.6; undergrad → 0.4
`fairness_deficit`	`fairness_ledger.fairness_term` for (member, resource_class); 0 if no history
`recency_penalty`	`min(1, confirmed_bookings_on_same_resource_last_7_days / 2)`
`academic_purpose`	1 if `purpose` keyword matches resource class affinity (e.g. "capstone" on computer lab; "thesis defence" on multimedia), else 0

Ties are broken deterministically by lower `member_id` UUID — order-independent and reproducible.

3.2 Conflict-Free Arbitration

The `bookings` table carries:

```
CONSTRAINT bookings_no_overlap EXCLUDE USING gist (
  resource_id WITH =,
  during     WITH &&
) WHERE (status = 'confirmed')
```

This constraint, backed by the `btree_gist` extension, makes it **physically impossible** to insert two confirmed bookings for the same resource with overlapping `tstzrange` values. No application-level check is required; the database engine enforces it.

`book_request` additionally calls `pg_advisory_xact_lock(hashtext(resource_id))` before any read-modify-write, serialising arbitration per resource. Two simultaneous requests for the same slot resolve in score order, not arrival order.

Flow inside `book_request`:

1. Idempotency check on `request_id` (safe retry).
2. Acquire advisory lock on the resource.
3. Check for an overlapping confirmed booking.
4. **No conflict** → insert confirmed booking, accrue served hours, write audit.
5. **Conflict exists** → score both parties.
   - Requester wins: demote incumbent to waitlist, confirm requester, audit `conflict_resolved`.
   - Incumbent wins: enqueue requester with their score and rank, audit `booking_waitlisted`.
6. In both cases return a `decision_explainer` JSON with winner, contenders, score components, and counterfactuals.

3.3 Fairness Rebalance Algorithm

`run_fairness_rebalance(window_days)` iterates over every `resource_class`:

```
For each resource class C over the past window_days:
  total_served   = SUM of booking hours for class C (confirmed + completed)
  active_members = COUNT of distinct members who booked class C in the window
  fair_share     = total_served / active_members

  For each active member M:
    served       = M's booking hours for class C in the window
    deficit      = fair_share − served
    fairness_term = GREATEST(0, LEAST(1, deficit / fair_share))
                  (over-served → 0; at fair-share → 0; maximally under-served → 1)

  UPSERT fairness_ledger(member, class, served_hours, fair_share, fairness_term)
```

The `fairness_term` is exactly the `fairness_deficit` component fed into the priority formula. Running rebalance after a period of skewed use immediately raises γ boosts for neglected users and zeroes them for heavy users, closing the gap in subsequent bookings.

3.4 Smart Waitlist Auto-Promote

`promote_top_waitlist(resource_id, during)` is called inside `cancel_booking` and `run_no_show_reaper`. It selects the `waiting` entry with the highest score for the freed slot and converts it to a confirmed booking atomically. No human intervention is needed; the slot never goes idle while a queued requester exists.

3.5 Swap Protocol

`propose_swap(actor, booking_a, booking_b)` implements a mutual-gain gate:

```
ua_b = score(A's member on A's slot)   — A's utility before swap
ua_a = score(A's member on B's slot)   — A's utility after swap
ub_b = score(B's member on B's slot)   — B's utility before swap
ub_a = score(B's member on A's slot)   — B's utility after swap

if ua_a < ua_b OR ub_a < ub_b → reject {ok:false, reason, deltas}
else → acquire locks on both resources, cancel both, re-insert swapped, audit
```

Neither side can be made worse off; the swap is only executed when it is strictly Pareto-improving.

3.6 Decision Explainer Contract

Every RPC that affects a booking returns an `explainer` field and writes it to `audit_log.decision_explainer`. The contract (abridged):

```
{
  "status": "confirmed_by_priority",
  "winner":    { "name": "Sarah Fernando", "score": 0.7257,
                "components": { "urgency": 1.0, "role_weight": 0.80,
                  "fairness_deficit": 0.619, "recency_penalty": 0.0, "academic_purpose": 1.0 } },
  "contenders":[ { "name": "Mihir Jain", "score": 0.5557, "rank": 1, "components": { … } } ],
  "counterfactuals": [ { "kind": "alternate_slot", "label": "Wed 14:00", "score": 0.61 },
                       { "kind": "alternate_resource", "label": "Lab-B", "score": 0.58 } ]
}
```

3.7 Worked Trace: Sarah vs. Mihir on Lab-A

Scenario: Mihir (year-1 undergrad) holds Lab-A 14:00–16:00 Tuesday. Sarah (final-year, capstone project, historically under-served) submits the same request.

Component	Mihir	Sarah
urgency (booking is ~24 h away)	1.00 (≤ 48 h boost caps urgency at 1.0)	1.00
role_weight	0.40 (undergrad)	0.80 (final-year)
fairness_deficit (γ)	0.619 (under-served on this class)	0.619 (under-served on this class)
recency_penalty	0.00	0.00
academic_purpose ("capstone")	0.00	1.00
total (α=0.25, β=0.30, γ=0.30, δ=0.10, ε=0.05)	**0.25×1.00 + 0.30×0.40 + 0.30×0.619 – 0 + 0 = 0.5557**	**0.25×1.00 + 0.30×0.80 + 0.30×0.619 – 0 + 0.05×1.00 = 0.7257**

Sarah's score (0.7257) beats Mihir's (0.5557). The two share the same `fairness_deficit` (0.619) because both have identical Lab-A history in the seed, so Sarah's margin comes entirely from role weight and academic-purpose match. `book_request` cancels Mihir's booking, enqueues him on the waitlist at rank 1, confirms Sarah, and writes the full explainer to the audit log. Sarah's Decision Modal shows "Confirmed by priority" with a bar chart of each component.

---

4. Architecture

Security model: RLS is enabled on all seven SYNAPSE tables. `SELECT` is granted to `anon` and `authenticated` (the availability feed is intentionally public). `INSERT`, `UPDATE`, and `DELETE` are revoked from all client roles. Clients can only mutate state through `SECURITY DEFINER` RPCs that accept an explicit `p_actor_id` and enforce role permissions inside the function body. This is server-authoritative regardless of whether the client is authenticated.

Realtime: Three tables publish to `supabase_realtime`. Client hooks subscribe to `postgres_changes` events on `bookings`, `waitlists`, and `audit_log`, keeping every screen live without polling.

Persona switcher: A Zustand store (persisted to `localStorage`) holds the active member. Every RPC call passes `actor_id = persona.id`. The production upgrade path is: replace `p_actor_id` with `auth.uid()` inside each RPC, re-enable Supabase Auth, and restore the middleware route guard — the auth wiring is already present in the repo from the base template.

5. Deviations from Phase 1 Proposal

These are honest refinements, not regressions. Judges are invited to view them as engineering judgement.

Phase 1 design	Prototype implementation	Why
Allocation logic in a **Deno Edge Function**	**Postgres SECURITY DEFINER RPC**	Runs in the same transaction as the write; no network hop; easier to test with SQL assertions; equally server-authoritative.
In-memory interval tree for conflict detection	**`tstzrange` + `EXCLUDE USING gist`** (btree_gist)	Double-booking becomes physically impossible, not just checked. Same O(log n) GiST behaviour. Bulletproof under concurrent transactions — no application-level gap.
Trigger.dev / pg_cron for scheduled rebalance and reaper	**Button-triggered ops** in the admin console	`pg_cron` is the production path (one `cron.schedule` call); omitted because it is not available in the Supabase local dev stack by default and adds no value to the demo. The SQL is trivial to add.
Real Supabase Auth + JWT-gated RLS	**Persona switcher** (no login)	The CIPHER rubric does not assess login. The auth flow from the base template is preserved in the repo — it was made dormant, not deleted. `p_actor_id` → `auth.uid()` is a one-line change per RPC.

6. Testing

Testing is a deliberate strength of this submission — it proves the engine is correct, not just visually plausible.

Suite	Count	Tool	What it covers
Unit tests	1 file	Vitest	Pure-function formatting helpers (`lib/synapse/__tests__/format.test.ts`)
RTL component tests	14 files	Vitest + React Testing Library	Every SYNAPSE component rendered and asserted: resource-card, resource-grid, slot-picker, booking-form, decision-modal, score-bars, my-bookings, persona-switcher, synapse-header, fairness-dashboard, policy-editor, audit-viewer, ops-panel, demo-control-room
Playwright E2E	5 specs / 10 tests	Playwright	Full browser flows: smoke (all pages load), booking (Mihir confirms; Sarah confirms by priority), contention (N-way simulate, winner + ranked contenders visible), admin (access gate; fairness tab; ops rebalance + reaper), my-bookings (cancel + promote)
SQL engine assertions	6 suites	psql	Direct Postgres assertions: schema constraints (exclusion constraint blocks overlap), helper functions, `priority_score` formula, `book_request` conflict paths, `simulate_contention` determinism, lifecycle RPCs (check-in, reaper, rebalance, swap)
Total	**155 Vitest + 10 Playwright + 6 SQL suites**		

The Playwright suite caught a real bug during development: the admin Ops panel called `run_fairness_rebalance` and `run_no_show_reaper` with a `p_actor_id` argument those functions do not accept, producing a PostgREST 404 at runtime. The mocked component tests passed (they asserted the call shape, not the real signature); only the end-to-end admin test, hitting real Postgres, surfaced it — before it reached the demo.

Run all suites: `pnpm test:all` (unit + component + SQL). Run E2E separately against a running dev server: `pnpm test:e2e`.

7. Limitations and Future Work

Limitation	Impact	Production path
No real authentication (persona switcher only)	Any client can impersonate any actor ID	Replace `p_actor_id` param with `auth.uid()` in each RPC; re-enable middleware guards; restore RLS policies that filter by `auth.uid()`
Scheduled ops are button-triggered	Reaper and rebalance must be run manually in the admin console	`SELECT cron.schedule('reaper', '*/15 * * * *', 'SELECT run_no_show_reaper()')` — one call per cron job once pg_cron is enabled on the Supabase project
No PWA / offline mode (F-02)	App requires connectivity	Add a Next.js service worker (next-pwa) for read-only cached views; write operations require connectivity by nature
No demand forecast (F-09)	Cannot predict peak demand	Implement Holt-Winters triple exponential smoothing over `bookings.during` time-series once sufficient history accumulates; the `fairness_ledger` window provides the baseline
Single-incumbent conflict only	`book_request` compares the requester against one conflicting booking (the earliest overlap); multi-incumbent arbitration is handled by `simulate_contention`	Extend `book_request` to collect all overlapping confirmed bookings and run the full multi-contender scoring loop from `simulate_contention`